

Challenge JDBC / JPA „IoT-Device“

Szenario: Ein Smart-Home-Hersteller möchte seine Geräte in einer PostgreSQL-Datenbank verwalten. Jedes Gerät besitzt eine eindeutige ID, einen Namen (z. B. "Thermostat Bad") und einen Typ (z. B. "Heizung").

Ihre Aufgabe: Implementieren Sie den gesamten Lebenszyklus eines Geräts (Anlegen der Tabelle, Speichern, Ändern und Löschen von Daten) auf zwei verschiedene Arten.

Teil A: Über JDBC

Nutzen Sie die Java Database Connectivity (JDBC) für die direkte Kommunikation.

1. **Verbindung:** Bauen Sie eine Verbindung zur Datenbank auf (Nutzen Sie Properties für User/Passwort).
2. **DDL:** Erstellen Sie die Tabelle `devices` mit `id INT PRIMARY KEY`, `name VARCHAR` und `type VARCHAR` mit einem JDBC Statement
3. **Sicheres Create:** Fügen Sie das Gerät "Klimaanlage" (Typ: "Kühlung") mit der ID 101 hinzu. Nutzen Sie zwingend ein **PreparedStatement**, um SQL-Injection zu verhindern.
4. **Sicheres Update:** Ändern Sie den Namen des ersten Geräts in "Haupt-Klimaanlage". Verwenden Sie auch hier ein **PreparedStatement**.
5. **Kontrolle** (Vor dem Löschen): Geben Sie alle Datensätze der Tabelle `devices` aus. Nutzen Sie `executeQuery` und eine `while(rs.next())`-Schleife.
6. **Sicheres Delete:** Löschen Sie das Gerät mit der ID 101 basierend auf seiner ID mittels **PreparedStatement**.
7. **Kontrolle (Nach dem Löschen):** Geben Sie erneut den gesamten Tabelleninhalt aus.
8. **Cleanup:** Schließen Sie alle Ressourcen ordnungsgemäß (`ResultSet`, `Statement`, `Connection`).

Teil B: Mit JPA/Hibernate (Objektorientiert)

Lösen Sie dasselbe Problem, indem Sie das Gerät als Java-Objekt (Entity) behandeln.

1. **Entity:** Erstellen Sie die Klasse `Device` mit den Annotationen `@Entity` und `@Id`. Verzichteten Sie hierbei explizit auf die Annotation `@GeneratedValue`, da die ID manuell vergeben werden soll. Achten Sie auf den erforderlichen Standard-Konstruktor.
2. **Persist:** Speichern Sie ein neues Device-Objekt mittels `em.persist()`. Vergeben Sie dabei im Java-Code explizit die **ID 101** für das Objekt. Dies muss innerhalb einer Transaktion geschehen.
3. **Update:** Laden Sie das Objekt mit `em.find()` und ändern Sie den Namen direkt über die Setter-Methode des Java-Objekts.

4. **Kontrolle (Vor dem Löschen):** Nutzen Sie JPQL (`SELECT d FROM Device d`), um alle Geräte zu laden und auszugeben
5. **Remove:** Markieren Sie das Objekt mit `em.remove()` zum Löschen.
6. **Transaktion:** Stellen Sie sicher, dass alle verändernden Operationen mit `em.getTransaction().begin()` gestartet und mit `commit()` abgeschlossen werden, damit die Änderungen dauerhaft in die Datenbank übernommen werden.

Beispiel-Lösung Teil A: Über JDBC

```
import java.sql.*;
import java.util.Properties;

public class Main {
    public static void main(String[] args) {
        // Verbindungsvorbereitung
        String url = "jdbc:postgresql://localhost:5432/postgres?currentSchema=public";
        //Tabelle wird im Schema public angelegt
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        try (Connection conn = DriverManager.getConnection(url, props)) {
            // 1. DDL Tabelle anlegen
            Statement st = conn.createStatement();
            st.executeUpdate("CREATE TABLE IF NOT EXISTS devices (id INT PRIMARY KEY, name
VARCHAR(255), type VARCHAR(255))");

            // 2. Datenanlage über PreparedStatement
            PreparedStatement pstmtIns = conn.prepareStatement("INSERT INTO devices (id, name,
type) VALUES (?, ?, ?)");
            pstmtIns.setInt(1, 101); // Manuelle ID
            pstmtIns.setString(2, "Klimaanlage");
            pstmtIns.setString(3, "Kühlung");
            pstmtIns.executeUpdate();

            // 3. Update mit PreparedStatement
            String updateSQL = "UPDATE devices SET name = ? WHERE id = ?";
            PreparedStatement pstmtUpd = conn.prepareStatement(updateSQL);
            pstmtUpd.setString(1, "Haupt-Klimaanlage");
            pstmtUpd.setInt(2, 1);
            pstmtUpd.executeUpdate();

            // 4. KONTROLLAUSGABE
            System.out.println("Inhalt vor dem Löschen:");
            ResultSet rs = st.executeQuery("SELECT * FROM devices");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") + " | Name: " +
rs.getString("name"));
            }

            // 5. Löschen mit PreparedStatement
            PreparedStatement pstmtDel = conn.prepareStatement("DELETE FROM devices WHERE id =
?");
            pstmtDel.setInt(1, 101);
            pstmtDel.executeUpdate();

            // 6. KONTROLLAUSGABE
            System.out.println("Inhalt nach dem Löschen:");
            rs = st.executeQuery("SELECT * FROM devices");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") + " | Name: " +
rs.getString("name"));
            }

            // Ressourcen schließen (S. 18)
            pstmtIns.close();
            pstmtUpd.close();
            pstmtDel.close();
            st.close();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Beispiel-Lösung Teil A: Über JDBC

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>JPA_Challenge_ManualID</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.2</version>
    </dependency>

    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.4.4.Final</version>
    </dependency>

    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.1.0</version>
    </dependency>
  </dependencies>
</project>
```

persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

  <persistence-unit name="JPACHallenge" transaction-type="RESOURCE_LOCAL">
    <class>org.example.Device</class>

    <properties>
      <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
      <property name="jakarta.persistence.jdbc.url"
value="jdbc:postgresql://localhost:5432/postgres"/>
      <property name="jakarta.persistence.jdbc.user" value="postgres"/>
      <property name="jakarta.persistence.jdbc.password" value="postgres"/>

      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>

      <property name="jakarta.persistence.schema-generation.database.action"
value="create"/>
    </properties>
  </persistence-unit>
</persistence>
```

Device.java

```
package org.example;

import jakarta.persistence.*;

@Entity
@Table(name = "devices") // Optionaler Tabellename
public class Device {

    @Id // Primärschlüssel-Variabale
    // WICHTIG: Kein @GeneratedValue, da wir die ID manuell setzen
    private Long id;

    private String name;
    private String type;

    // Standard-Konstruktor: Für JPA zwingend erforderlich!
    public Device() {
    }

    // Getter und Setter
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getType() { return type; }
    public void setType(String type) { this.type = type; }

    // Hilfreich für die Konsolenausgabe
    @Override
    public String toString() {
        return "Device{id=" + id + ", name='" + name + "', type='" + type + "'}";
    }
}
```

Main.java

```
package org.example;

import jakarta.persistence.*;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        // Name muss exakt der persistence-unit aus der persistence.xml entsprechen!
        EntityManagerFactory factory = Persistence.createEntityManagerFactory("JPACHallenge");
        EntityManager em = factory.createEntityManager();

        try {
            // --- 1. SCHRITT: Speichern (Persist) ---
            em.getTransaction().begin();

            Device dev = new Device();
            dev.setId(101L); // MANUELLE ID vergeben
            dev.setName("Klimaanlage");
            dev.setType("Kühlung");

            em.persist(dev); // Objekt in den Persistence Context aufnehmen
            em.getTransaction().commit();
            System.out.println("Gerät erfolgreich gespeichert.");

            // --- 2. SCHRITT: Ändern (Update) ---
            em.getTransaction().begin();
            Device managedDev = em.find(Device.class, 101L); // Objekt über ID finden
            if (managedDev != null) {
                managedDev.setName("Haupt-Klimaanlage"); // Update direkt am Objekt
            }

            // --- 3. SCHRITT: Kontrolle vor dem Löschen (JPQL) ---
            System.out.println("---- Inhalt vor dem Löschen ----");
        }
    }
}
```

```

        // JPQL nutzt Klassennamen 'Device', nicht Tabellennamen 'devices'!
        List<Device> allDevices = em.createQuery("SELECT d FROM Device d",
Device.class).getResultList();
        allDevices.forEach(System.out::println);
        em.getTransaction().commit();

        // --- 4. SCHRITT: Löschen des Datensatzes (Remove) ---
        em.getTransaction().begin();
        if (managedDev != null) {
            em.remove(managedDev); // Objekt zur Löschung markieren
        }

        // --- 5. SCHRITT: Tabelle physisch löschen (Cleanup) ---
        System.out.println("Lösche die gesamte Tabelle 'devices'...");
        em.createNativeQuery("DROP TABLE devices").executeUpdate(); // Native SQL für DDL

        em.getTransaction().commit();
        System.out.println("Tabelle und Daten erfolgreich entfernt.");

    } catch (Exception e) {
        if (em.getTransaction().isActive()) {
            em.getTransaction().rollback(); // Bei Fehler alles rückgängig machen
        }
        e.printStackTrace();
    } finally {
        // Ressourcen freigeben
        em.close();
        factory.close();
    }
}

```